

TetrioCoach: 대규모 리플레이 데이터 기반 적응형 테트리스 AI 코칭 시스템

TetrioCoach: An Adaptive Tetris AI Coaching System
Based on Large-Scale Replay Data Analysis

이창호 (LeeChangHo)

무소속 독립 연구자 (Independent Researcher)

초록 (Abstract)

본 연구는 경쟁형 퍼즐 게임 TETR.IO의 리플레이 데이터를 활용한 적응형 AI 코칭 시스템 TetrioCoach를 제안한다. 기존 테트리스 AI 연구가 봇의 게임 성능 극대화에 집중한 반면, 본 시스템은 인간 플레이어의 약점을 자동 분류하고 수준별 맞춤형 훈련 피드백을 생성하는 처방적(prescriptive) 코칭 파이프라인을 구축하였다. 상위 500명 플레이어의 61,935매치(7,716,524 배치) 실제 데이터를 K-Means 클러스터링으로 라벨링하고 GradientBoosting으로 학습한 하이브리드 분류기는 Macro F1-score 0.960을 달성하였으며, 16개 빌드 패턴과 11개 랭크 티어의 교차 추천 매트릭스를 통해 플레이어 수준에 따라 차별화된 전략적 조언을 생성한다. 4,087줄의 완전한 시스템은 외부 LLM 의존성 없이 독립적으로 동작하며, 기술적(descriptive) 통계를 넘어 처방적 피드백으로의 전환이라는 새로운 연구 방향을 제시한다.

키워드: 테트리스 AI, 자동 코칭, 리플레이 분석, 약점 분류, 기계학습, e스포츠, 처방적 피드백, 빌드 패턴

AI 활용 고지 (AI Disclosure Statement)

본 연구의 시스템 설계, 코드 구현, 문헌 조사 및 논문 구조화 과정에서 Anthropic사의 대규모 언어 모델 Claude를 보조 도구로 활용하였다. Claude는 코드 작성 보조, 학술 문헌 검색, 논문 개요 구조 제안 등에 사용되었으며, 모든 연구 설계 의사결정, 실험 설계, 결과 해석 및 학술적 주장의 최종 판단은 저자 이창호가 수행하였다. Claude는 연구의 책임성과 법적 주체성을 가질 수 없으므로 저자 명단에 포함되지 않으며, IEEE/ACM/Nature 등 주요 학술지의 AI 사용 공개 가이드라인을 준수하여 본 고지를 통해 투명하게 공개한다.

I. 서론 (Introduction)

1.1 연구 배경 및 동기

경쟁형 퍼즐 게임 테트리스는 1984년 Alexey Pajitnov에 의해 개발된 이래 40년간 전 세계적으로 가장 널리 플레이되는 비디오 게임 중 하나로 자리잡았다. 특히 온라인 대전 플랫폼 TETR.IO는 2026년 기준 950만 이상의 등록 사용자와 10억 회 이상의 누적 게임을 기록하며, 테트리스의 경쟁적 측면이 e스포츠 생태계의 주요 영역으로 부상하고 있음을 보여준다.

그러나 플레이어의 실력 향상을 체계적으로 지원하는 코칭 시스템은 학술적으로나 산업적으로 미개척 상태이다. 인간 코치에 의존하는 기존 방식은 확장성이 제한적이며, 최근 부상한 LLM(대규모 언어 모델) 기반 코칭은 환각(hallucination) 문제와 API 비용, 비결정적 출력 등의 한계를 내포한다.

1.2 연구 목적 및 질문

본 연구는 다음 세 가지 연구 질문에 답하고자 한다:

- RQ1: 대규모 리플레이 데이터에서 플레이어의 약점 유형을 자동으로 분류할 수 있는가?
- RQ2: 분류된 약점에 기반한 맞춤형 피드백이 플레이어 수준별로 차별화될 수 있는가?
- RQ3: 도메인 특화 지식(빌드 패턴, 티어 벤치마크)의 구조화가 피드백 품질을 향상시키는가?

1.3 학술적 기여

본 연구의 핵심 기여(Hook)는 다음과 같다:

"기존 테트리스 AI 연구(Dellacherie, 2003; Thiery & Scherrer, 2009; Gabillon et al., 2013)는 '봇이 얼마나 잘 두는가'에 집중하였으나, 본 연구는 '인간 플레이어를 얼마나 효과적으로 가르치는가'라는 근본적으로 다른 질문에 답한다."

- Descriptive-to-Prescriptive 전환: 통계 기술(記述)을 넘어, ML 기반 약점 분류 -> 티어별 벤치마크 비교 -> 맞춤형 훈련 처방 파이프라인 구축
- 다층 데이터 추상화: 7.7M 배치 단위 데이터를 6단계 추상화를 거쳐 자연어 코칭으로 변환
- 도메인 지식 구조화: 16개 빌드 패턴 x 11개 랭크 티어의 교차 추천 매트릭스 최초 구축

II. 이론적 배경 (Theoretical Background)

2.1 테트리스 AI의 학술적 계보

테트리스는 NP-완전 문제로 증명되었으며(Breukelaar et al., 2004), 10x20 보드의 가능한 상태 수는 2^{200} 에 달하여 완전한 MDP 해법이 불가능하다(Bodoia & Puranik, 2012). 이에 따라 휴리스틱 기반 접근법이 주류를 이루어 왔다.

Dellacherie(2003)는 landing height, row/column transitions, holes, wells, eroded cells의 6개 특성으로 구성된 선형 평가 함수를 제안하여 평균 660,000줄 클리어를 달성하였으며, Thiery & Scherrer(2009)는 Cross Entropy 기법으로 이를 검증하고 개선하였다. Gabillon et al.(NeurIPS, 2013)은 근사 동적 프로그래밍으로 이를 초월하였으며, Cold Clear(MinusKelvin, 2019)는 유전 알고리즘을 통해 20+개 특성의 가중치를 진화시켜 현대 테트리스 봇의 사실상 표준이 되었다. Erkalkan(2026)은 GA 기반 동적 가중치 조정으로 고정 가중치 대비 +61.79%의 성능 향상을 보고하였다.

2.2 e스포츠 AI 코칭 연구

게임 AI를 플레이어 코칭에 활용하는 연구는 최근 급격히 성장하고 있다. Google DeepMind의 TacticAI(2024)는 축구 전술 분석에 ML을 적용하였으며, Jing et al.(2025)은 SHAP 기반 해석 가능 ML로 Counter-Strike 프로 선수의 12년 성과를 분석하였다. 그러나 리플레이 데이터에서 자동으로 약점을 분류하고, 도메인 특화 지식과 결합하여 처방적 피드백을 생성하는 통합 프레임워크는 학술적으로 체계화되지 않은 상태이다.

2.3 기계학습 기반 플레이어 프로파일링

Gradient Boosting(Friedman, 2001)은 약한 학습기의 앙상블을 순차적으로 구성하여 분류 및 회귀 성능을 극대화하는 기법으로, 정형 데이터에서 높은 성능을 보인다. 본 연구에서는 실제 게임 데이터에서 추출한 10개 특성 벡터를 기반으로 플레이어의 주요 약점 유형(speed, attack, tspin, finesse, defense, balanced)을 분류하는 데 활용한다.

III. 연구 방법 (Methodology)

3.1 시스템 아키텍처

TetrioCoach는 4계층 구조로 설계되었다: (L1) 데이터 수집층 - TETR.IO API, 로컬 .ttrm 파일, Kaggle 데이터셋, 빌드 패턴 DB; (L2) 통계 분석층 - 집계 엔진, 보드 시뮬레이터, 플레이 스타일 분석기, 빌드 감지기; (L3) AI 추론층 - ML 약점 분류기, 규칙 기반 평가기, 휴리스틱 보드 평가기; (L4) 피드백 생성층 - 코칭 리포트, 훈련 로드맵, 빌드 어드바이저, 매치업 조언.

[Fig. 1: TetrioCoach Multi-Layer Architecture 삽입 위치]

3.2 데이터 수집 및 전처리

데이터는 세 가지 경로로 수집된다. (1) TETR.IO 공개 API(ch.tetr.io/api)를 통한 리더보드 및 전적 조회 - APM, PPS, VS, 가비지 송수신 등 요약 통계 제공. (2) 로컬 .ttrm 파일의 직접 파싱 - replay.results.stats 경로에서 T-Spin 유형별(TSS/TSD/TST/Mini), 라인 클리어, Finesse, 배치 수 등 상세 통계 추출. (3) Kaggle 공개 데이터셋(n3koasakura, 2024) - 상위 500명 플레이어의 61,935매치에서 7,716,524개 배치 단위 데이터 활용. 각 배치에 보드 상태(playfield), T-Spin 유형, 콤보, B2B, 가비지, Glicko-2 레이팅이 포함된다.

3.3 통계 집계 엔진

수집된 라운드 데이터에서 30+개의 메트릭을 산출한다: 승률(매치/라운드), 평균 APM/PPS/VS, T-Spin 유형별 횟수 및 비율, 라인 클리어 분포(Single/Double/Triple/Quad), Finesse Fault 비율, 가비지 공방비(pressure ratio), 최대 콤보/B2B/스파이크, APM/PPS 추이(트렌드). 온라인 API와 로컬 ttrm의 데이터

가용성 차이를 has_detail 플래그로 관리하여, UI와 AI 분석 모두에서 데이터 부재를 명시적으로 처리한다.

3.4 ML 약점 분류기

본 연구의 ML 분류기는 순환 논리(Circular Reasoning)를 회피하기 위해 2단계 접근을 채택한다. (1단계) K-Means 클러스터링($k=8$)으로 20,000게임의 10차원 특성 벡터에서 자연적 군집을 도출한다. 사용된 10개 특성은 다음과 같다: (1) PPS, (2) APM 추정치($\text{attack/piece} \times 60$), (3) T-Spin율(%), (4) Quad율(%), (5) 피스당 라인(lines_per_piece), (6) 최대 콤보, (7) 최대 B2B, (8) 최대 보드 높이, (9) 수비 비율(가비지 클리어/수신), (10) $\text{rating_norm}(\text{TR}/1000)$. 각 군집의 프로파일(위 특성들의 전체 평균 대비 $z\text{-score}$)을 기반으로 약점 라벨을 매핑한다. 이를 통해 입력 변수에 대한 하드코딩 규칙이 아닌, 데이터 자체의 구조에서 라벨이 도출된다. (2단계) 클러스터 라벨로 GradientBoostingClassifier($n\text{-estimators}=200$, $\text{max_depth}=6$)를 학습하여 새로운 플레이어의 약점을 예측한다.

추론 시에는 ML 확률(가중치 0.4)과 규칙 기반 점수(가중치 0.6)를 결합한 하이브리드 예측을 사용한다. 이는 학습 데이터(상위 500명)와 일반 플레이어 간의 스케일 불일치를 보완하기 위함이다. 클래스 불균형에 대해서는 노이즈 주입 오버샘플링($\text{min_ratio}=0.10$)을 적용하였으며, 최종 성능은 단일 Accuracy가 아닌 Macro-averaged F1-score로 보고한다.

2단계 파이프라인(K-Means $\rightarrow$ GradientBoosting)의 설계 근거: 본 구조는 K-Means의 기하학적 결정 경계를 지도학습 모델로 대리 학습(Surrogate Learning)하는 것으로, 높은 F1-score 자체는 수학적으로 예상 가능한 결과이다. 그러나 2단계 구조를 채택한 실익은 다음과 같다: (1) K-Means는 새 데이터에 대해 가장 가까운 중심까지의 거리만으로 이산 라벨을 할당하지만, GradientBoosting은 확률 분포(predict_proba)를 제공하여 경계선 사례에서 소프트 판단이 가능하다. (2) 하이브리드 추론 시 ML 확률과 규칙 기반 점수를 가중 결합하는 과정에서, 연속 확률값을 제공하는 지도학습 모델이 이산 클러스터 할당보다 유연한 융합을 가능하게 한다. (3) K-Means 재실행 없이 사전 학습된 분류기로 즉시 추론이 가능하며, GUI 앱의 실시간 응답($<100\text{ms}$)에 적합한 연산 효율을 확보한다.

[Table 1] ML 약점 분류기 성능 (K-Means 라벨링 + 오버샘플링)

Class	Precision	Recall	F1	Support
attack	0.97	0.97	0.97	1,293
defense	0.96	0.97	0.96	1,106
speed	0.96	0.96	0.96	1,017
tspin	0.95	0.93	0.94	584
Macro avg	0.96	0.96	0.96	4,000

3.5 빌드 패턴 지식 베이스

Hard Drop Wiki 및 FOUR.lol에서 수집한 16개 빌드 패턴(TKI, DT Cannon, PCO, Hachispin, Albatross, MKO, C-Spin, STSD, LST Stacking, Imperial Cross, Fractal, 4-wide, 6-3 Stacking, Downstacking 등)을 구조화된 데이터베이스로 구축하였다. 각 빌드에 카테고리($\text{opener/midgame/strategy/defense}$), 필요 백 수, 공격 유형, 출력 라인, 요구 조건, 커버리지, 난이도, 적용 티어, 강점/약점/카운터 전략을 포함하며, $C \sim X+$ 11개 티어별 최적 빌드 추천 매트릭스를 구성하였다.

빌드-티어 매핑의 객관성을 확보하기 위해, 각 빌드의 적용 티어는 (1) Hard Drop Wiki 및 FOUR.lol에 명시된 빌드별 난이도·요구 조건, (2) 해당 빌드가 실전에서 통용되는 TR 구간에 대한 커뮤니티 통계 및 오픈소스 가이드(예: 4-wide는 중급 구간에서 효과적이나 상위 구간에서는 카운터 당함, Albatross/Fractal 등 연속 T-Spin 빌드는 $X+$ 구간에서 주로 사용)를 교차 참조하여 결정하였다. 즉, 매트릭스의 매핑 규칙은 저자의 임의적 판단이 아니라 공개된 도메인 지식과 커뮤니티 합의에 근거한다. 다만 이 매핑의 정량적 타당성에 대한 전문가 패널 검증은 향후 과제로 남는다.

3.6 피드백 생성 파이프라인

피드백 생성은 다음 단계로 진행된다: (1) `compute_aggregates_v2()`로 통계 집계 -> (2) `build_profile_from_agg()`로 `PlayerProfile` 구성 -> (3) `evaluate_profile()`로 `WeaknessReport` 리스트 생성 (6개 카테고리, 4단계 심각도, 우선순위 점수) -> (4) `predict_weakness()`로 ML 약점 예측 -> (5) 코칭 리포트(8개 섹션) + 훈련 로드맵(티어별 20분 루틴) + 매치업 조언(상대 빌드 대응) 생성. 전 과정이 외부 API 호출 없이 로컬에서 100ms 이내에 완료된다.

IV. 연구 결과 (Results)

4.1 ML 모델 성능

K-Means 클러스터링 기반 라벨링과 오버샘플링 적용 후, GradientBoosting 분류기는 테스트셋(n=4,000)에서 Accuracy 0.962, Macro F1-score 0.960, 5-fold CV Macro F1 0.956(+/-0.003)을 달성하였다. 본 연구는 클래스 불균형 환경에서 단일 Accuracy가 다수 클래스에 의해 과대평가되는 문제를 회피하기 위해 Macro F1-score를 일차 평가 지표로 채택한다. 오버샘플링으로 4개 클래스(attack, defense, speed, tspin) 간 Support가 584~1,293으로 균형화(기존 극단적 불균형 12~12,908에서 개선)되어, Accuracy(0.962)와 Macro F1(0.960)이 0.002 이내로 근접하였다. 이는 모든 클래스에서 분류 성능이 고르게 확보되었음을 의미하며, 두 지표 간 괴리가 작다는 점이 균형화의 성공을 방증한다.

Feature importance 분석 결과, lines_per_piece(28.6%)가 가장 높은 중요도를 보였으며, rating_norm(21.7%), max_btb(15.9%), tspin_rate(6.4%)가 뒤를 이었다. 이전 버전에서 tspin_rate(73.9%)가 압도적이었던 것은 규칙 기반 라벨링의 순환 논리에 기인한 것으로, 비지도 클러스터링 기반에서는 라인 효율과 전체 레이팅이 약점 구분의 핵심 변수로 나타났다. 이는 모델이 단일 지표가 아닌 다차원적 특성 조합으로 약점을 판별함을 의미한다.

rating_norm 변수에 관한 주석: 본 연구에서 rating_norm은 Kaggle 데이터셋의 TR(Tetra Rating) 값을 1000으로 나눈 스케일 정규화 변수이다(상위 500명 기준 평균 약 24.9). 학습 데이터가 X+ 단일 구간에 집중되어 표준편차가 0.095로 작아, 학습 단계에서는 군집 간 미세한 레이팅 차이를 보조 신호로 활용하나, 신규 플레이어 추론 시에는 해당 플레이어의 TR을 알 수 없으므로 학습 평균값(24.9)으로 고정하여 입력한다. 따라서 rating_norm의 21.7% 중요도는 학습 분포 내에서의 기여이며, 실제 추론에서는 개별 플레이어를 구분하는 변별 변수가 아닌 집단 수준의 앵커(anchor)로 작용한다. 이 점은 전 티어 데이터 확보 시 rating_norm을 실측값으로 대체하여 변별력을 회복할 수 있는 향후 개선 지점이다.

[Fig. 2: Feature Importance Bar Chart 삽입 위치]

4.2 기존 접근법 대비 비교

[Table 2] 기존 테트리스 AI 접근법과의 비교

연구	목적	방법	출력
Dellacherie (2003)	최적 플레이	6-feature 휴리스틱	배치 위치
Gabillon et al. (2013)	최적 플레이	ADP 가치 함수	배치 위치
Cold Clear (2019)	최적 플레이	GA 가중치 진화	배치 위치
Erkalkan (2026)	최적 플레이	GA 동적 가중치	배치 위치
TetrioCoach (본 연구)	인간 코칭	ML 분류 + 규칙 + 도메인 지식	자연어 피드백

4.3 사례 연구

사례 1: 플레이어 종단 성장 추이 분석 (Subject P, 1년간 37매치)

동일 플레이어의 1년간 리플레이 데이터(37개 .ttrm 파일)를 초기(2025-06~08), 중기(2025-12~2026-03), 최근(2026-06) 세 구간으로 분할하여 분석하였다.

[Table 4] 플레이어 Subject P의 종단 성장 추이 (1년간)

구간	Rounds	APM	PPS	승률	T-Spin %	Fault%	ML 예측
초기 (2025-06~08)	31	49.3	1.81	54.8%	4.1%	61.3%	attack (48%)

중기 (2025-12~2026-03)	41	45.9	1.75	56.1%	3.9%	58.3%	attack (52%)
최근 (2026-06)	16	55.1	1.92	62.5%	4.3%	55.8%	finesse (56%)

1년간 APM이 49.3에서 55.1로, PPS가 1.81에서 1.92로, 승률이 54.8%에서 62.5%로 상승하였다. Finesse Fault는 61.3%에서 55.8%로 개선되었으나 여전히 심각 수준이다. 주목할 점은 하이브리드 ML 예측이 초기/중기에는 "attack"(0.48~0.52)을 1순위로, 최근에는 "finesse"(0.56)로 전환된다는 것이다(2순위 약점도 attack↔finesse가 교차). 이는 APM이 49.3에서 55.1로 향상되며 공격력 약점이 상대적으로 해소되는 한편, Finesse가 새로운 핵심 병목으로 부상하는 성장 패턴을 모델이 포착한 것으로 해석된다. 동일 피험자의 종단 데이터에서 약점 진단이 시간에 따라 이동한다는 점은, 모델이 단순히 고정된 라벨로 수렴하지 않고 입력 지표 변화에 반응함을 보여준다.

사례 2: 고수준 균형형 플레이어 분석 (Player A, 익명)

PPS 2.36(고속), APM 54.1, T-Spin율 4.33%, Finesse Fault 6.8%(우수)로 모든 핵심 지표가 균형적이며 정확도가 특히 뛰어나다. 하이브리드 ML 모델은 "balanced"를 1순위(확신도 0.58)로 예측하였고, 규칙 기반 평가기에서도 finesse 약점 점수가 거의 0에 수렴하였다. 추정 티어 S 기준으로 TKI, DT Cannon, Hachispin, STSD를 추천하며, "현재 수준을 유지하면서 다음 티어를 목표"라는 유지 관리형 피드백을 생성하였다. 이는 시스템이 뚜렷한 약점이 없는 균형형 플레이어를 올바르게 식별함을 보여준다.

사례 3: 저속-T-Spin 미활용형 플레이어 분석 (Player C, 익명)

PPS 1.56(저속), APM 50.9, T-Spin율 1.55%(낮음), Finesse Fault 17.9%(양호). 하이브리드 ML 모델은 "speed"를 1순위(확신도 0.58)로 예측하였으며, 규칙 기반 평가기 또한 낮은 PPS를 핵심 약점으로 검출하였다. 정확도(Fault 17.9%)는 상대적으로 양호하나 속도와 T-Spin 활용이 부족한 패턴으로, 시스템은 추정 티어 A 기준 TKI/DT Cannon/STSD 추천과 함께 20분 루틴 첫 단계에 "40L 스프린트 3회 — 속도 감각 워밍업"을 우선 배치하여 속도 개선을 최우선으로 하는 처방을 생성하였다.

사례 4: 다중 플레이어 비교 분석

[Table 5] 4명 플레이어의 다중 비교 분석 (검증 가능 데이터)

지표	Subject P	Player A	Player B	Player C
PPS	1.83	2.36	1.62	1.56
APM	51.0	54.1	53.9	50.9
T-Spin%	4.82	4.33	4.54	1.55
Fault%	57.7	6.8	36.6	17.9
추정 티어	S	S	A+	A
ML 1순위	attack/finesse	balanced	attack	speed
ML 확신도	0.46/0.46	0.58	0.41	0.58
처방 초점	정확도+공격	유지관리	공격 빌드	속도

서로 다른 지표 프로필을 가진 4명의 플레이어에 대해, 하이브리드 ML 모델은 4가지 상이한 1순위 약점을 예측하였다: Player A(Fault 6.8%로 정확도 우수, PPS 2.36)는 "balanced"(0.58), Player B(Fault 36.6%, 중속)는 "attack"(0.41), Player C(PPS 1.56로 저속, T-Spin 1.55%)는 "speed"(0.58), Subject P(Fault 57.7%)는 "attack/finesse"가 0.46으로 동률을 이루었다. 예측 결과가 입력 프로필의 차이(정확도-속도-T-Spin 활용도)에 따라 명확히 분화함을 확인할 수 있다.

약점 수렴에 대한 오류 분석 (Error Analysis): 본 연구의 예비 분석에서, 동일한 좁은 스킬 구간(S~SS)에서 Finesse Fault가 유사하게 높은(54~58%) 플레이어들만을 대상으로 했을 때 예측이 finesse로 수렴하는 현상이 관찰되었다. 이는 모델 자체의 편향이라기보다, 하이브리드 예측에서 규칙 기반 점수(가중치 0.6)가

지배적인 구조에서 입력군의 공통 약점이 그대로 반영된 결과이다. [Table 5]와 같이 정확도·속도·T-Spin 활용도가 이질적인 플레이어들을 입력하면 예측이 balanced/attack/speed/finesse로 정상 분화하므로, 수렴 현상은 입력 표본의 동질성에 기인한 것임을 확인하였다. 다만 규칙 기반 가중치(0.6)가 ML(0.4)보다 높아 경계선 사례에서 규칙 점수가 우세한 구조적 특성이 존재하며, 전 티어 학습 데이터 확보 시 ML 가중치를 상향하여 데이터 주도 분류를 강화할 수 있다.

최종 처방의 질적 대조: ML 1순위가 다른 플레이어 간에는 후속 파이프라인의 처방이 명확히 갈린다. "balanced"로 판정된 Player A에게는 20분 루틴 첫 단계로 "TKI 오프닝 패턴 10회 반복"(공격 빌드 숙련)이, "speed"로 판정된 Player C에게는 "40L 스프린트 3회 — 속도 감각 위밍업"이 우선 배치되었다. 또한 추정 티어(S/S+/A+/A)에 따라 빌드 추천 집합이 달라져, Player A-B에게는 PCO·Hachispin 등 고난도 오프닝이, Player C에게는 STSD 중심의 기초 빌드가 제시되었다. 이로써 ML 라벨 → 티어 추정 → 빌드 매트릭스 → 약점 우선순위의 조합이 플레이어별로 질적으로 상이한 훈련 처방을 생성함을 실증하였다.

사례 5: S~U 티어 벤치마크 대비 Subject P 성장 분석

TETR.IO 공개 리더보드 API에서 S, S+, SS, U 각 티어의 플레이어 10명을 무작위 추출하여 평균 통계를 산출하고, Subject P의 종단적 성장 궤적과 비교하였다.

[Table 6] S~U 티어 벤치마크 대비 Subject P 위치 비교

구간/티어	TR	APM	PPS	VS	승률	n
S tier avg	14,996	39.7	1.45	86.5	45.3%	10
S+ tier avg	16,276	42.9	1.43	96.7	24.8%	10
SS tier avg	17,861	58.0	1.74	121.6	30.7%	10
U tier avg	20,163	79.9	2.05	167.0	33.5%	10
Subject P 초기	-	49.3	1.81	108.7	54.8%	31R
Subject P 중기	-	45.9	1.75	94.1	56.1%	41R
Subject P 최근	-	55.1	1.92	119.4	62.5%	16R

Subject P의 최근 지표(APM 55.1, PPS 1.92, VS 119.4)는 SS 티어 평균(APM 58.0, PPS 1.74, VS 121.6)과 근접하며, PPS에서는 SS 평균을 초과한다. 반면 APM은 SS 평균에 약간 미달하여, 시스템이 진단한 "공격 전환 효율"이 다음 성장 포인트임을 티어 벤치마크가 객관적으로 뒷받침한다. 1년간의 성장 궤적은 S 수준(초기)에서 SS 근접(최근)으로의 상승을 보여주며, 이는 TetrioCoach의 피드백이 제시하는 개선 방향과 일치하는 간접 검증 근거가 된다.

V. 논의 (Discussion)

5.1 학술적 의의

본 연구는 (1) 테트리스 리플레이 데이터의 다층 추상화 프레임워크, (2) ML 기반 약점 자동 분류의 실증, (3) 도메인 지식 구조화(빌드 패턴 x 티어)의 피드백 품질 기여를 입증함으로써, 리플레이 기반 자동 코칭이라는 새로운 연구 영역을 개척하였다.

5.2 한계점

- ML 학습 데이터 편향: 상위 500명(X+ 랭크) 집중으로, 모델의 특성 분포가 일반 플레이어와 스케일 불일치를 보임. 이를 보완하기 위해 하이브리드 예측(ML 40% + 규칙 60%)을 적용하였으나, 전 티어 데이터 확보 시 ML 비중을 높일 수 있음
- 라벨링 방법론: K-Means 클러스터링이 규칙 기반 라벨링의 순환 논리를 개선하였으나, 인간 전문가(코치)의 교차 검증을 거친 골드 스탠다드 라벨과의 비교 검증이 필요함
- 보드 시뮬레이터: DAS/ARR 타이밍 불일치로 라인 클리어 재현 정확도 제한

- 하이브리드 예측 구조: 규칙 기반 가중치(60%)가 ML(40%)보다 높아, 분석 대상 그룹의 공통 약점(높은 Finesse Fault)이 있을 경우 동일 라벨로 수렴하는 경향이 관찰됨. 전 티어 학습 데이터 확보 시 ML 비중을 높여 개선 가능
- 사용자 연구(User Study) 부재: 처방적 피드백의 실효성을 정량적으로 검증하기 위해 최소한 소규모 파일럿 테스트(티어별 2~3명, TR 변화 추적)가 필요하나 본 연구에서는 수행하지 못함. 본 논문의 사례 분석은 간접적 사후적 정합성 검증에 해당함

5.3 윤리적 고려사항

본 연구의 사례 분석에 사용된 리플레이 데이터의 플레이어들에게 개별 동의를 구하지 않았다. 이에 따라 논문 내 저자 본인을 포함한 모든 플레이어의 닉네임은 Subject P, Player A, Player B, Player C 등으로 익명화 처리하였다. ML 모델 학습에 사용된 Kaggle 데이터셋(n3koasakura, 2024)은 공개 데이터셋으로, TETR.IO의 공개 API를 통해 수집된 데이터이며 개인 식별 정보를 포함하지 않는다. 향후 사용자 연구 수행 시에는 IRB(기관생명윤리위원회) 승인 절차를 거칠 것을 명시한다.

5.4 향후 연구

- Cold Clear 2(Rust) 직접 빌드를 통한 배치 정확도 비교 시스템 완성
- TETR.IO 봇 계정 확보를 통한 전 티어 리플레이 수집 및 모델 재학습
- A/B 사용자 연구: TetrioCoach 사용 그룹 vs 미사용 그룹의 TR 변화 추적
- 실시간 코칭: 게임 중 보드 상태 캡처를 통한 즉시 피드백 시스템 확장
- Puyo Puyo, 리듬 게임 등 타 경쟁 게임으로의 프레임워크 일반화

VI. 결론 (Conclusion)

본 연구는 테트리스 AI 코칭이라는 새로운 연구 영역에서, 대규모 리플레이 데이터 분석 -> ML 기반 약점 분류 -> 도메인 지식 기반 처방적 피드백 생성이라는 완전한 파이프라인을 설계, 구현하고 사례 분석을 통해 시스템의 일관성을 간접 검증하였다. 4,087줄의 시스템은 외부 LLM 없이 독립적으로 동작하며, 61,935매치 실제 데이터를 K-Means 클러스터링으로 라벨링하고 학습한 분류기는 Macro F1-score 0.960을 달성하였다. 16개 빌드 패턴과 11개 티어의 교차 추천 매트릭스를 통해 초급부터 최상위까지 모든 수준의 플레이어에게 맞춤형 코칭을 제공한다. 이 프레임워크는 테트리스를 넘어 리플레이 기반 경쟁 게임의 자동 코칭에 일반화될 수 있으며, Descriptive에서 Prescriptive로의 전환이라는 AI 코칭의 새로운 패러다임을 제시한다.

참고 문헌 (References)

- [1] Algorta, S. & Simsek, O. (2019). The Game of Tetris in Machine Learning. arXiv:1905.01652.
- [2] Bodoia, M. & Puranik, A. (2012). Applying Reinforcement Learning to Competitive Tetris. Stanford CS229 Project Report.
- [3] Breukelaar, R., Demaine, E.D., Hohenberger, S., Hoogeboom, H.J., Kusters, W.A., & Liben-Nowell, D. (2004). Tetris is Hard, Even to Approximate. *International Journal of Computational Geometry & Applications*, 14(1-2), 41-68.
- [4] Dellacherie, P. (2003). Algorithm for Tetris. Unpublished manuscript.
- [5] Erkalkan, E. (2026). Heuristic Optimization of a Tetris Bot Using Genetic Algorithms: An Adaptive Evolutionary Approach. *Turkish Journal of Mathematics and Computer Science*.
- [6] Friedman, J.H. (2001). Greedy Function Approximation: A Gradient Boosting Machine. *Annals of Statistics*, 29(5), 1189-1232.
- [7] Gabillon, V., Ghavamzadeh, M., & Scherrer, B. (2013). Approximate Dynamic Programming Finally Performs Well in the Game of Tetris. *Advances in Neural Information Processing Systems (NeurIPS)*, 26, 1754-1762.
- [8] Jing, S. et al. (2025). Interpretable Machine Learning with SHAP for Esports Performance Analysis of Professional Counter-Strike Players from 2012 to 2025. *International Journal of Sports Science & Coaching*.
- [9] MinusKelvin. (2019). Cold Clear: Tetris Bot. GitHub. <https://github.com/MinusKelvin/cold-clear>
- [10] n3koasakura. (2024). Tetr.io Top Players Replays (Tetris). Kaggle Dataset. <https://www.kaggle.com/datasets/n3koasakura/tetr-io-top-players-replays>
- [11] Stevens, M. (2016). Playing Tetris with Deep Reinforcement Learning. Stanford CS231N Reports.
- [12] Thiery, C. & Scherrer, B. (2009). Improvements on Learning Tetris with Cross Entropy. *International Computer Games Association Journal*, 32(1), 23-33.

부록 A: 시각 자료 기획 가이드

A.1 Fig. 1 - 시스템 아키텍처 흐름도

4계층(Data Acquisition -> Statistical Analysis -> AI Intelligence -> Feedback Generation) 구조. 각 계층에 소속 모듈과 데이터 흐름 표시. 본 논문 개요 작성 시 생성된 SVG 다이어그램을 기반으로 고해상도 벡터 이미지로 제작.

A.2 Fig. 2 - Feature Importance 분석

(a) 수평 바 차트: 10개 특성의 중요도 순위. lines_per_piece(28.6%), rating_norm(21.7%), max_btb(15.9%) 순으로 강조. 이전 버전의 tspin_rate(73.9%) 과적합과 대비하여 다차원 분포로 개선되었음을 시각적으로 제시. (b) Confusion Matrix(4x4): attack/defense/speed/tspin. (c) ROC Curve: multi-class, one-vs-rest.

A.3 Fig. 3 - 피드백 생성 파이프라인

입력(ttrm/API) -> 파싱 -> 집계 -> [ML 분류기 + 규칙 평가기 + 빌드 DB] -> 코칭 + 로드맵 + 매치업. 각 단계의 데이터 변환과 추상화 수준을 시각화.

A.4 Table 3 - 빌드 패턴 x 티어 추천 매트릭스

16개 빌드(행) x 11개 티어(열)의 교차표. 해당 티어에서 추천되는 빌드에 체크 표시. 빌드별 공격 유형, 난이도, 출력 라인 병기.

A.5 Fig. 4 - 사용자 대시보드 스크린샷

(a) 성장 그래프 탭: 6개 subplot (APM, PPS, VS, 가비지, T-Spin/Quad, Finesse). (b) 통계 요약 탭: 온라인/로컬 분기 표시. (c) AI 코칭 탭: 8개 섹션 피드백 텍스트. (d) 훈련 로드맵 탭: 20분 루틴.